Programming Language Principles, Design, and Implementation

Compilers - Abstract Machines

Vincent Rahli

University of Birmingham

Where are we?

- Part 1: Principles of programming
- **Part 2: Compilers**

Today

Abstract Machines

- Call-by-name
- Call-by-value

Further reading:

- *"A Functional Correspondence between Evaluators and Abstract Machines"* by Mads Sig Ager, Dariusz Biernacki, Olivier Danvy, and Jan Midtgaard
- *"The Zinc Experiment: An Economical Implementation Of The Ml Language"* by Xavier Leroy

Recap: Krivine's Abstract Machine

Last lecture we ended up with the following abstract machine, which is for the call-by-name λ-calculus:

$$
\begin{array}{lcl}
\mathtt{asm}(x, e, k) & = & \mathtt{asm}(M, e', k) \\
\quad \text{where } e(x) = \langle M, e' \rangle & & \\
\mathtt{asm}(\lambda x.M, e, []) & = & \langle \lambda x.M, e \rangle \\
\mathtt{asm}(\lambda x.M, e, \langle N, e' \rangle :: k) & = & \mathtt{asm}(M, e[x \mapsto \langle N, e' \rangle], k) \\
\mathtt{asm}(M\ N, e, k) & = & \mathtt{asm}(M, e, \langle N, e \rangle :: k)
\end{array}
$$

which is of type $(\mathtt{LC} \times \mathtt{Env} \times \mathtt{Cont}) \to (\mathtt{LC} \times \mathtt{Env})$

This **abstract state machine** (ASM)

- **pushes** a closure to the stack every time it sees an **application**
- **pops** a closure from the stack when it sees a λ-**abstraction**, to perform a β-reduction, storing the argument in the heap

Krivine's Abstract Machine

Let us step away from the λ-calculus.

First, we want a more machine-amenable version of the λ-calculus using **de Bruijn indices**:

$$M ::= n \mid \lambda M \mid M\ M$$

where n is a number, which points the n'th λ going up the parse tree.

For example:

$$\begin{array}{lll} \lambda x.\lambda y.x & \text{is} & \lambda\lambda 1 \\ \lambda x.\lambda y.y & \text{is} & \lambda\lambda 0 \end{array}$$

Call-by-name β-reduction is now defined as follows:

$$(\lambda M)N \mapsto_\beta M[0 \backslash N]$$

where

$$\begin{array}{lcl} (M\ N)[n\backslash P] & = & M[n\backslash P]\ N[n\backslash P] \\ (\lambda M)[n\backslash P] & = & \lambda(M[n{+}1 \backslash \Uparrow_0 P]) \\ n[n\backslash P] & = & P \\ m[n\backslash P] & = & m, \text{ if } m \neq n \end{array} \quad \left| \quad \begin{array}{lcl} \Uparrow_n (M\ N) & = & \Uparrow_n M\ \Uparrow_n N \\ \Uparrow_n \lambda M & = & \lambda \Uparrow_{n+1} M \\ \Uparrow_n m & = & m+1, \text{ if } m \geqslant n \\ \Uparrow_n m & = & m, \text{ if } m < n \end{array} \right.$$

Krivine's Abstract Machine

Exercise: convert $(\lambda x.\lambda y.x)\ (\lambda z.z)$ into the de Bruijn notation, and reduce the expression.

$(\lambda x.\lambda y.x)\ (\lambda z.z)$ is $(\lambda\lambda 1)\ \lambda 0$ in de Bruijn notation.

It reduces as follows:

- $(\lambda\lambda 1)\ \lambda 0 \mapsto_\beta (\lambda 1)[0\backslash\lambda 0]$
- $= \lambda(1[1\backslash\lambda 0])$
- $= \lambda\lambda 0$

which corresponds to $\lambda y.\lambda z.z$ as expected.

Krivine's Abstract Machine

Krivine's abstract machine is now:

$$
\begin{array}{lcl}
\texttt{asm}(0, \langle M, e' \rangle :: e, k) & = & \texttt{asm}(M, e', k) \\
\texttt{asm}(n{+}1, \langle M, e' \rangle :: e, k) & = & \texttt{asm}(n, e, k) \\
\texttt{asm}(\lambda M, e, []) & = & \langle \lambda M, e \rangle \\
\texttt{asm}(\lambda M, e, \langle N, e' \rangle :: k) & = & \texttt{asm}(M, \langle N, e' \rangle :: e, k) \\
\texttt{asm}(M\ N, e, k) & = & \texttt{asm}(M, e, \langle N, e \rangle :: k)
\end{array}
$$

which is of type $(\texttt{LC} \times \texttt{Env} \times \texttt{Cont}) \rightarrow (\texttt{LC} \times \texttt{Env})$
where Env is now $\texttt{list}(\texttt{LC} \times \texttt{Env})$

This machine has 3 instructions:

- **pushes** a closure to the stack every time it sees an **application**
- **pops/grabs** a closure from the stack when it sees a λ-**abstraction**, to perform a β-reduction, storing the argument in the heap
- **accesses** a closure from the heap when it sees a variable

Krivine's Abstract Machine

We introduce a 3-instruction language:

- $\texttt{push}(c)$: to push an instruction onto the stack
- $\texttt{grab}$: to move a closure from the stack to the heap
- $\texttt{access}(n)$: to access a closure from the heap
- plus sequencing: $c_1; c_2$

We convert the λ-calculus to this language as follows:

$$\begin{array}{lcl} [\![n]\!] & = & \texttt{access}(n) \\ [\![\lambda M]\!] & = & \texttt{grab}; [\![M]\!] \\ [\![M\ N]\!] & = & \texttt{push}(N); [\![M]\!] \end{array}$$

Krivine's abstract machine is now:

$$\begin{array}{lcl} K(\texttt{access}(0); c, \langle c', e' \rangle :: e, k) & = & K(c', e', k) \\ K(\texttt{access}(n{+}1); c, \langle c', e' \rangle :: e, k) & = & K(n, e, k) \\ K(\texttt{grab}; c, e, []) & = & \langle \texttt{grab}; c, e \rangle \\ K(\texttt{grab}; c, e, \langle c', e' \rangle :: s) & = & K(c, \langle c', e' \rangle :: e, s) \\ K(\texttt{push}(c'); c, e, k) & = & K(c, e, \langle c', e \rangle :: k) \end{array}$$

Krivine's Abstract Machine - Example

Convert the λ-expression $((\lambda x.\lambda y.y)\ M\ N)$ to a sequence of instructions, and evaluate it

- de Bruijn notation: $(\lambda\lambda 0)\ M'\ N'$ where M' is the de Bruijn version of M, and similarly for N
- $[\![(\lambda\lambda 0)\ M'\ N']\!] = \mathtt{push}(c_N); \mathtt{push}(c_M); \mathtt{grab}; \mathtt{grab}; \mathtt{access}(0)$, where $c_M = [\![M']\!]$ and $c_N = [\![N']\!]$
- $$\begin{aligned} & K(\mathtt{push}(c_N); \mathtt{push}(c_M); \mathtt{grab}; \mathtt{grab}; \mathtt{access}(0), [], []) \\ &= K(\mathtt{push}(c_M); \mathtt{grab}; \mathtt{grab}; \mathtt{access}(0), [], [\langle c_N, []\rangle]) \\ &= K(\mathtt{grab}; \mathtt{grab}; \mathtt{access}(0), [], [\langle c_M, []\rangle, \langle c_N, []\rangle]) \\ &= K(\mathtt{grab}; \mathtt{access}(0), [\langle c_M, []\rangle], [\langle c_N, []\rangle]) \\ &= K(\mathtt{access}(0), [\langle c_N, []\rangle, \langle c_M, []\rangle], []) \\ &= K(c_N, [], []) \end{aligned}$$

Note: a k-ary function pushes k closures onto the heap

Krivine's Abstract Machine – Correctness

Let us show the correctness of this abstract machine.

We first convert instructions back to λ-expressions as follows:

$$\begin{array}{lcl}
\overline{\texttt{access}(n); c} & = & n \\
\overline{\texttt{grab}; c} & = & \lambda \overline{c} \\
\overline{\texttt{push}(c_2); c_1} & = & \overline{c_1}\ \overline{c_2}
\end{array}$$

Stacks and Heaps are converted as follows:

$$\overline{[(c_0, e_0), \ldots, (c_n, e_n)]} = \overline{c_0}[0 \backslash \overline{e_0}]\ \cdots\ \overline{c_n}[0 \backslash \overline{e_n}]$$
$$\overline{[]} = 0$$

A triple (c, e, k) is converted as follows:

$$\overline{(c, e, [(c_1, e_1), \ldots, (c_n, e_n)])} = \overline{[(c, e), (c_1, e_1), \ldots, (c_n, e_n)]}$$

Krivine's Abstract Machine – Correctness

Correctness theorem: if $K(c_1, e_1, k_1)$ computes to $K(c_2, e_2, k_2)$ then $\overline{(c_1, e_1, k_1)}$ reduces to $\overline{(c_2, e_2, k_2)}$ under the call-by-name semantics.

For example:

- $K(\mathtt{push}(c_N); \mathtt{push}(c_M); \mathtt{grab}; \mathtt{grab}; \mathtt{access}(0), [], []) =$
  $K(\mathtt{grab}; \mathtt{access}(0), [\langle c_M, [] \rangle], [\langle c_N, [] \rangle])$
- $\overline{\mathtt{push}(c_N); \mathtt{push}(c_M); \mathtt{grab}; \mathtt{grab}; \mathtt{access}(0)} = (\lambda\lambda 0)\ \overline{c_M}\ \overline{c_N}$
- $\overline{(\mathtt{push}(c_N); \mathtt{push}(c_M); \mathtt{grab}; \mathtt{grab}; \mathtt{access}(0), [], [])} = (\lambda\lambda 0)\ \overline{c_M}\ \overline{c_N}$
- $\overline{\mathtt{grab}; \mathtt{access}(0)} = \lambda 0$
- $\overline{(\mathtt{grab}; \mathtt{access}(0), [\langle c_M, [] \rangle], [\langle c_N, [] \rangle])} = (\lambda 0)[0 \backslash \overline{c_M}]\ \overline{c_N} = (\lambda 0)\ \overline{c_N}$

The SECD call-by-value machine

Created by **Landin** in 64

Stands for: **S**tack, **E**nvironment, **C**ontrol, **D**ump

For simplicity, we consider this extension of the λ-calculus (the first n is a variable, the second is a number):

$$M ::= n \mid \lambda M \mid M\ M \mid \underline{n} \mid M + M$$

Instructions of the (Modern) SECD machine:

- `const`(n): stores a number on the stack
- `add`: adds 2 numbers stored on the stack
- `access`(n): moves n^{th} entry from the environment to the stack
- `closure`(c): pushes a closure onto the stack
- `apply`: pops a function & argument from the stack, and performs an application
- `return`: terminates a function and jumps back to caller

The SECD call-by-value machine

We convert the λ-calculus to this language as follows:

$$\begin{array}{lcl}
[\![n]\!] & = & \texttt{access}(n) \\
[\![\lambda M]\!] & = & \texttt{closure}([\![M]\!]; \texttt{return}) \\
[\![M\ N]\!] & = & [\![M]\!]; [\![N]\!]; \texttt{apply} \\
[\![\underline{n}]\!] & = & \texttt{const}(n) \\
[\![M + N]\!] & = & [\![M]\!]; [\![N]\!]; \texttt{add}
\end{array}$$

Environment & Stack:

- The environment now store values (the arguments)
- The stack stores both values and closures

The SECD abstract machine is as follows:

$$\begin{array}{lcl}
SECD(\texttt{access}(n); c, e, s) & = & SECD(c, e, \texttt{lookup}(e, n) :: s) \\
SECD(\texttt{closure}(c'); c, e, s) & = & SECD(c, e, \langle c', e \rangle :: s) \\
SECD(\texttt{apply}; c, e, v :: \langle c', e' \rangle :: s) & = & SECD(c', v :: e', \langle c, e \rangle :: s) \\
SECD(\texttt{return}; c, e, x :: \langle c', e' \rangle :: s) & = & SECD(c', e', x :: s) \\
SECD(\texttt{const}(n); c, e, s) & = & SECD(c, e, n :: s) \\
SECD(\texttt{add}; c, e, v_1 :: v_2 :: s) & = & SECD(c, e, (v_1 + v_2) :: s)
\end{array}$$

The SECD call-by-value machine – Example

Convert the λ-expression $((\lambda x.\lambda y.y)\ (\underline{1}+\underline{2})\ \underline{4})$ to a sequence of instructions, and evaluate it. We will see that $(\underline{1}+\underline{2})$ gets reduced.

- $$\begin{aligned} c_0 &= \texttt{closure}(\texttt{access}(0);\texttt{return});\texttt{return} \\ c_1 &= \texttt{closure}(c_0) \\ c_2 &= \texttt{const}(1);\texttt{const}(2);\texttt{add};\texttt{apply};\texttt{const}(4);\texttt{apply} \\ c &= c_1;c_2 \end{aligned}$$
- $$\begin{aligned} &SECD(c,[],[]) \\ &= SECD(c_2,[],[\langle c_0,[]\rangle]) \\ &= SECD(\texttt{const}(2);\texttt{add};\texttt{apply};\texttt{const}(4);\texttt{apply},[],[1,\langle c_0,[]\rangle]) \\ &= SECD(\texttt{add};\texttt{apply};\texttt{const}(4);\texttt{apply},[],[2,1,\langle c_0,[]\rangle]) \\ &= SECD(\texttt{apply};\texttt{const}(4);\texttt{apply},[],[\boxed{3},\langle c_0,[]\rangle]) \\ &= SECD(c_0,[3],[\langle \texttt{const}(4);\texttt{apply},[]\rangle]) \\ &= SECD(\texttt{return},[3],[\langle \texttt{access}(0);\texttt{return},[3]\rangle,\langle \texttt{const}(4);\texttt{apply},[]\rangle]) \\ &= SECD(\texttt{const}(4);\texttt{apply},[],[\langle \texttt{access}(0);\texttt{return},[3]\rangle]) \\ &= SECD(\texttt{apply},[],[4,\langle \texttt{access}(0);\texttt{return},[3]\rangle]) \\ &= SECD(\texttt{access}(0);\texttt{return},[4,3],[\langle \epsilon,[]\rangle]) \\ &= SECD(\texttt{return},[4,3],[4,\langle \epsilon,[]\rangle]) \\ &= SECD(\epsilon,[],[4]) \end{aligned}$$

The SECD call-by-value machine – Correctness

The correctness of the SECD machine is significantly harder to prove than the one of the Krivine machine.

Correctness theorem: if $SECD([\![M]\!], [], [])$ computes to $SECD(\epsilon, [], [[\![V]\!]])$ then M reduces to V under the call-by-value semantics.

Krivine vs. SECD

Why didn't we add numbers to the language on which we defined the Krivine machine?

Because operations such as addition are strict (they require evaluating all arguments), which requires additional mechanisms and complicates the abstract machine.

SECD β-reduction:

$$\begin{aligned}
&SECD(\texttt{closure}(c_1;\texttt{return});c_2;\texttt{apply},e,s)\\
&= SECD(c_2;\texttt{apply},e,\langle c_1;\texttt{return},e\rangle :: s)\\
&= \cdots = SECD(\texttt{apply},e,v :: \langle c_1;\texttt{return},e\rangle :: s)\\
&= SECD(c_1;\texttt{return},v :: e,\langle \epsilon,e\rangle :: s)
\end{aligned}$$

where c_1 is the body of the λ-abstraction and c_2 the argument

Krivine β-reduction:

$$\begin{aligned}
&K(\texttt{push}(c_2);\texttt{grab};c_1,e,s)\\
&= K(\texttt{grab};c_1,e,\langle c_2,e\rangle :: s)\\
&= K(c_1,\langle c_2,e\rangle :: e,s)
\end{aligned}$$

Krivine vs. SECD

In the case of a function with multiple parameters:

$$(\lambda \cdots \lambda M)\ N_1\ \cdots\ N_k$$

The K machine will push all k arguments onto the stack, and "grab" them by moving them to the heap (environment).

The SECD machine will do something like this $k-1$ times:

$$\begin{aligned}
&SECD(\texttt{apply}; c_2; \texttt{apply} \cdots ; c_k; \texttt{apply}, e', v :: \langle \texttt{closure}(c); \texttt{return}, e \rangle :: s) \\
&= SECD(\texttt{closure}(c); \texttt{return}, v :: e, \langle c_2; \texttt{apply} \cdots ; c_k; \texttt{apply}, e' \rangle :: s) \\
&= SECD(\texttt{return}, v :: e, \langle c, v :: e \rangle :: \langle c_2; \texttt{apply} \cdots ; c_k; \texttt{apply}, e' \rangle :: s) \\
&= SECD(c_2; \texttt{apply} \cdots ; c_k; \texttt{apply}, e', \langle c, v :: e \rangle :: s)
\end{aligned}$$

which pushes a short-lived closure onto the stack.

This can be avoided using for example the **ZINC** machine (not covered in this module).

Conclusion

What did we cover today?

- Abstract machines
- Call-by-name
- Call-by-value